

AI and the Reinvention of Performance Reviews in Enterprise Software Engineering

Executive summary

AI-assisted development is collapsing the usefulness of “volume” proxies (lines of code, commit counts, PR counts) as signals of individual performance. Controlled experiments show AI tools can increase speed and throughput on coding tasks and workplace task completion—often without a corresponding increase in real engineering impact, and with meaningful risk of quality and security regressions if verification is weak. ¹

The strategic implication for engineering managers is not to “audit prompts,” but to redesign performance reviews around observable, difficulty-adjusted contributions: how an engineer frames ambiguous problem spaces, makes technical tradeoffs, reduces risk, collaborates to unblock others, and ships maintainable outcomes. Research on developer productivity measurement consistently warns that productivity cannot be reduced to a single metric; activity measures can reflect brute force, tooling friction, or the ease of the work rather than value created. ²

A practical, lower-effort path is to (a) define difficulty bands for work (ambiguity, dependencies, blast radius, novelty), (b) replace volume metrics with a small set of performance dimensions, each anchored in “good traces” and “bad traces,” and (c) run a lightweight, AI-assisted improvement (PIP-style) workflow that auto-collects evidence from systems of record (tickets, PRs, CI, incidents) while keeping humans in the loop and minimizing invasive surveillance. This aligns with established performance management guidance that emphasizes clear objectives, regular discussions, and documented improvement plans—while avoiding algorithmic-management pitfalls where workers are primarily “evaluated by the system.” ³

Why traditional volume proxies break under AI-assisted development

AI changes the production function of code

In controlled experiments, access to GitHub Copilot significantly reduced time-to-complete a programming task (about 55.8% faster in one controlled setup). ⁴ Real-world randomized trials across Microsoft, Accenture, and an anonymous Fortune 100 company report an estimated ~26% increase in completed tasks for those using the AI tool, alongside increases in commits and builds—i.e., more *visible activity* becomes cheaper to generate. ⁵

Implication: When “output tokens” (code, diffs, commits) are cheaper to produce, they stop differentiating effort, judgment, or impact. A developer can appear “productive” by pushing large volumes of AI-generated code even if they are not effectively solving the underlying problem or are creating downstream cost (review burden, regressions, security debt).

Activity metrics were always fragile—AI makes them actively misleading

Developer productivity research has long cautioned that productivity cannot be reduced to a single dimension, and that the “productivity is all about activity” myth causes undesirable outcomes and dissatisfaction. Higher activity can signal brute forcing through poor systems or planning rather than effectiveness. ⁶

Similarly, the “No Single Metric Captures Productivity” chapter (open access) highlights how LOC-based measurement is “clearly fraught,” encourages gaming, and fails to capture quality, maintainability, and the wide variety of valuable work developers do beyond writing code. ⁷ This is a classic Goodhart’s Law setup: the more a metric becomes a target, the more it gets optimized (and gamed) rather than measuring what you care about. ⁸

AI increases verification burden and risk costs—often invisibly

Early field/usage research on Copilot emphasizes that developers can spend more time reviewing than writing, and that roles shift toward assessing suggestions rather than “doing the task.” ⁹ Security research finds substantial fractions of AI-generated suggestions can be vulnerable in certain scenarios (e.g., ~40% vulnerable in one Copilot security analysis), and user studies find developers with AI assistance may write less secure code while *believing* it is secure. ¹⁰

NIST’s Generative AI risk profile also highlights **automation bias** and **over-reliance**: humans may over-trust AI outputs and perceive them as higher quality, which can exacerbate risks like confabulation and bias. ¹¹

Implication: If you keep rewarding volume, you may inadvertently reward engineers who externalize costs to reviewers, SRE, security, and future maintainers.

A difficulty-adjusted performance model for AI-era engineering work

A workable replacement for volume metrics is a **difficulty-adjusted scorecard**. The goal is not to turn performance into a single number, but to make differences in *problem difficulty* explicit, then evaluate performance against observable dimensions. This aligns with measurement best practice: define goals first, then questions, then metrics (Goal-Question-Metric). ¹²

Step one: normalize for difficulty using a lightweight “difficulty band”

For each meaningful deliverable (ticket/epic/incident/project), assign a **Difficulty Band** using four observable factors:

Difficulty factor	What it means in practice	Low	Medium	High	Extreme
Ambiguity	Unclear requirements, shifting scope, discovery work	Clear spec	Some unknowns	Major discovery	Ill-defined / R&D

Difficulty factor	What it means in practice	Low	Medium	High	Extreme
Dependencies	Cross-team reliance, external approvals, vendor/tool constraints	None	1–2 known	Multiple / contested	Unowned / blocking
Blast radius	Risk if wrong: customer impact, security, compliance, reliability	Local	Service-level	Multi-service	Platform / prod-wide
Novelty	New domain, new stack, new architecture	Routine	Some new	New subsystem	First-of-kind

You don't need perfection—just consistency and transparency so “unlucky hard problems” don't get evaluated like “routine work with AI autocomplete.”

Step two: evaluate performance on observable, difficulty-adjusted dimensions

Use a small set of dimensions that remain meaningful even when code is cheap.

Technical judgment (tradeoffs & decision quality). Evidence that the engineer chooses appropriate solutions, anticipates failure modes, and adapts when new information arrives.

Ownership & delivery reliability. Whether they move work to “done” responsibly: clarify requirements, manage dependencies, communicate status, and close loops.

Quality & maintainability. Whether they produce changes that are testable, readable, reviewable, and stable—not just voluminous.

Risk mitigation (security, privacy, reliability). Whether they reduce the probability/impact of failure through testing, threat modeling, safe rollout, and monitoring.

Collaboration & leverage. Whether they unblock others, review effectively, mentor, and make the team faster—not just themselves.

These map naturally to the SPACE framework's warning against over-focusing on “Activity” and its insistence on multidimensional productivity. ¹³

Step three: tie the model to organizational outcomes (without weaponizing metrics)

Use delivery and reliability metrics as *context*, not as the single grade. DORA's current guidance defines software delivery performance metrics (evolved beyond the original “four keys”) and frames them as indicators of delivery outcomes; DORA also explicitly documents metric evolution and definitions. ¹⁴

Critically, DORA's 2025 AI-focused research message is that **AI is an amplifier** (magnifying strengths/weaknesses); the biggest gains come from the underlying system (culture, platform, data ecosystem), not the tool alone. ¹⁵

What to look for: good traces and bad traces

The manager's problem ("lazy vs unlucky") is best solved by looking for **process evidence that correlates with effective engineering**—the traces that appear when someone is doing the hard thinking and responsible delivery work, even when the code was AI-assisted.

Good traces that strongly predict "hard-working but stuck on a hard problem"

Clear problem framing before heavy coding.

Look for a short written artifact (ticket comment, doc, PR description) that states: hypothesis, constraints, non-goals, and a plan to de-risk. This is the behaviors that measurement frameworks recommend: start with the question/goal, then instrument. ¹²

Evidence of de-risking early.

Examples: a small spike PR, feature flag scaffolding, incremental rollout plan, added monitoring, or a test harness. NIST's GAI risk guidance highlights the need to manage human-AI configuration risks and to document and measure risks rather than implicitly trusting outputs. ¹⁶

High-signal collaboration.

Examples: asking precise questions of partner teams, summarizing decisions, proposing interfaces, and unblocking reviews. GitHub describes PR reviews as a primary collaboration mechanism to ensure code quality and share knowledge. ¹⁷

Reviewability discipline.

Small or logically segmented PRs; PR descriptions that help reviewers; prompt response to review feedback. Copilot can generate PR summaries, which can reduce manager time spent extracting intent—if engineers edit them into accurate narratives. ¹⁸

Quality behaviors that don't show up in LOC.

Meaningful tests, refactors that reduce complexity, docs, runbooks, and operational readiness. This is precisely what LOC misses: quality, maintainability, and the non-coding work that creates organizational value. ⁷

Bad traces that predict "low effort hidden by AI output"

Lots of code, little convergence.

Patterns: repeated reverts, thrash, high churn without explanation, or "big bang" PRs that are hard to review and repeatedly fail CI.

Low verification posture.

Patterns: minimal tests, missing edge cases, ignoring static analysis/security alerts. Security research shows

AI suggestions can be insecure and that users can be overconfident in AI-assisted code—making verification an explicit performance expectation. ¹⁹

Passive dependency management.

Patterns: long blocked periods with no escalation, no alternative plan, or no proactive communication. This is one of the clearest separators between “unlucky” and “low effort”: unlucky engineers surface blockers early and propose paths around them.

“Rubber-stamping” collaboration.

Patterns: performative reviews, shallow comments, slow response to teammates, or failing to follow through on commitments.

Unowned operational consequences.

Patterns: repeated incidents/regressions tied to their changes without postmortem participation, weak rollbacks, or lack of monitoring. DORA’s emphasis on delivery and reliability outcomes underscores that shipping is not the end—operating safely is part of performance. ²⁰

A fast heuristic: “narrative + artifacts”

Ask: can the engineer produce a coherent narrative of what happened, what they tried, what they learned, and what they will do next—with *artifacts to back it up* (PRs, tickets, CI runs, incident timelines)?

NIST’s explainability principles emphasize that for a system (and by extension, AI-assisted work) to be trustworthy, it needs explanations that are meaningful, accurate, and bounded by knowledge limits. Applying this to humans: the best engineers can explain what the AI did, what they accepted/rejected, and why. ²¹

An AI-assisted improvement workflow that is low-effort and defensible

This section is **not legal advice**; it is a pragmatic workflow designed to reduce manager effort while aligning with widely recognized HR best practices: clear expectations, regular check-ins, documented support, and fair opportunity to improve. ²²

It also avoids turning performance management into algorithmic management—where tracked data becomes the manager—an approach the ILO flags as increasingly common and consequential. ²³

Design principles

Use systems of record, not prompt surveillance. Evidence should come from work artifacts that already exist for delivery (tickets, PRs, CI, incidents), not from logging private “how they think” data.

Human-in-the-loop for every evaluative step. NIST’s GAI profile explicitly warns about automation bias and over-reliance; your process should force human verification of AI-generated summaries and conclusions. ¹¹

Bias and privacy safeguards by default. NIST notes bias risk is socio-technical and cannot be reduced to zero; build guardrails (calibration, second reviewer, consistent rubric) rather than assuming neutrality. ²⁴

Workflow overview (mermaid)

```
graph TD
    A[Define expectations & difficulty band] --> B[Auto-collect evidence pack weekly]
    B --> C[AI drafts weekly summary + deltas vs goals]
    C --> D[Manager reviews & corrects summary]
    D --> E[Weekly checkpoint: feedback + remove blockers + reset next week goals]
    E --> F{On track?}
    F -->|Yes| G[Continue plan / close early if sustained]
    F -->|No| H[Escalate supports + tighten scope]
    H --> I{Pass criteria met by end date?}
    I -->|Yes| J[Close plan + monitor]
    I -->|No| K[HR consult + decision (role change / exit)]
```

The “Evidence Pack” (automated, manager-verified)

Your AI assistant (or internal tooling) should produce a **weekly packet** with:

- Work list (tickets/epics owned, difficulty band, status, next step)
- PR list (opened/merged, size, review comments, time-to-first-review, rework indicators)
- CI results for their PRs (pass rate, flakes, reruns)
- Post-merge signals (rollbacks, hotfixes, incidents connected to their changes)
- Collaboration signals (reviews given, review turnaround, unblocking interactions)

This is consistent with the idea that Copilot usage metrics and PR telemetry can help understand adoption and workflow effects, but you should not treat AI-usage telemetry as “performance” by itself. ²⁵

Objective pass/fail criteria (difficulty-adjusted)

Define criteria that are **independent of raw code volume**:

Delivery (adjusted by difficulty band).

- Low/Medium: reliably deliver planned milestones; minimal reopen/rework.
- High/Extreme: demonstrate weekly de-risking progress (validated hypotheses, prototypes, clarified requirements, dependency movement), even if final ship is later.

Quality.

- No recurring “same-class” defects; required tests added/updated; CI green on merge; no avoidable rollbacks.
- If security scanning exists: alerts are triaged promptly; no repeated high-severity issues. (Use tooling to enforce, not manager memory.) ²⁶

Collaboration.

- Meets review/response SLAs; escalates blockers within X days; communicates status weekly.

Judgment.

- Produces a short decision log for significant tradeoffs; can explain why AI suggestions were accepted or rejected.

Lean 30/60/90 AI-assisted PIP template (sample)

Scope note: keep it narrow; pick one role's core outcomes, not personality traits.

Timebox	Goals (difficulty-adjusted)	Required behaviors (good traces)	Evidence sources	Pass criteria
0–30 days	Stabilize execution on 1–2 medium-scope deliverables	Weekly plan + risk list; small PRs; tests for changed code; prompt review response	Tickets, PRs, CI, review threads	≥80% of weekly commitments met or documented de-risking progress; CI pass on merges; no repeatable avoidable regressions
31–60 days	Deliver one end-to-end feature or subsystem change with safe rollout	Design note; feature flag/rollout checklist; monitoring; incident readiness	Design doc, PRs, release notes, dashboards	Feature shipped (or staged per plan); post-merge stability acceptable; on-call burden not increased due to preventable issues
61–90 days	Demonstrate sustained ownership and team leverage	Mentors/reviews others; proactively unblocks; maintains quality bar	Review logs, teammate feedback, metrics	Sustained performance at role level; manager and peer signals align; no “volume-only” pattern

Comparing options: manual PIP vs AI-assisted PIP vs immediate termination

This is a decision support table, not legal guidance.

Option	Manager effort	Legal/HR risk (general)	Fairness & accuracy	Effectiveness
Manual PIP (no automation)	High	Medium	Medium (depends on documentation quality)	Medium

Option	Manager effort	Legal/HR risk (general)	Fairness & accuracy	Effectiveness
AI-assisted PIP (evidence pack + human review)	Medium–Low	Lower–Medium (better documentation, consistency)	Higher (less memory bias; clearer criteria)	High (faster feedback cycles)
Immediate termination	Low initially	Highest (documentation gaps; consistency questions)	Lowest	Variable (fast, but risky; may fail to address systemic causes)

Tooling and integrations that support the workflow

The goal is to **raise the signal-to-noise ratio** of performance evidence while reducing manager toil.

Repository workflow enforcement (quality gates)

Use GitHub’s built-in guardrails to make “quality behaviors” the default:

- **CODEOWNERS** to define responsibility and route reviews to owners. ²⁷
- **Protected branches / rulesets** to require PRs, approvals, and required status checks. ²⁸
- **Code scanning with CodeQL** to surface security issues automatically (default setup is available). ²⁹
- **Secret scanning + push protection** to prevent secret leaks before they land. ³⁰

These shift performance conversations from “I think you’re lazy” to “the system requires X; you repeatedly did not meet X.”

AI assistants for review and summarization (not for judgment-by-telemetry)

- **Copilot PR summaries:** Generate a starting description for reviewers; require the author to edit for accuracy. ¹⁸
- **Copilot code review:** Use as a first-pass reviewer to flag issues; managers should treat it as an assistant, not an authority. ³¹
- **Copilot Autofix for code scanning:** For CodeQL alerts, provides suggested fixes and can reduce remediation friction. ³²

Telemetry and metrics aggregation (team outcomes; individual evidence packs)

- **Copilot usage metrics dashboard/API:** Useful for adoption and workflow analysis; note it is derived from telemetry, requires telemetry enabled, and excludes some surfaces (e.g., Copilot Chat on GitHub.com and GitHub Mobile). ³³
- **DORA tooling:** DORA’s resources list source-available tools to collect and visualize delivery metrics (e.g., Apache DevLake, OpenDORA). ³⁴
- Apache DevLake documentation includes DORA dashboards and troubleshooting/validation dashboards, which can help debug metric definitions rather than blindly trust them. ³⁵

Privacy, data minimization, and bias controls (configure explicitly)

NIST's GAI profile recommends privacy-aware handling of data (e.g., remove PII, anonymize, consider privacy-enhancing technologies, and support consent/withdrawal in human-subject contexts). While this is published for GAI systems, it translates directly into "don't over-collect employee data." ³⁶

If you are tempted to log prompts: treat prompts as potentially sensitive content. Prefer **artifact-based evidence** over "thought surveillance," and keep any AI summaries auditable and correctable by humans (NIST explainability principles: meaningful explanations, explanation accuracy, knowledge limits). ³⁷

Manager playbook: calibration, scripts, and low-effort documentation

Calibration steps that reduce "lazy vs unlucky" errors

1. **Pre-calibrate difficulty bands** across your team for the quarter (agree what "high ambiguity" looks like). This directly addresses confounding factors, which measurement research highlights as a core reason single metrics mislead. ³⁸
2. **Define two levels of expectations:** "table stakes" (quality gates, communication cadence) and "impact" (difficulty-adjusted outcomes).
3. **Use a second-reader review** for any adverse action: another manager reviews the evidence pack and rubric scores to detect bias or inconsistency (NIST: bias is socio-technical and not fully eliminable; build governance). ²⁴
4. **Document consistency:** apply the same rubric and gates to everyone; exceptions should be explicit (e.g., incident response week).

Communication scripts (short, usable)

Resetting expectations (before any formal plan)

"With AI tools, code volume is no longer a reliable signal. Going forward, I'll evaluate performance based on delivery outcomes adjusted for difficulty, quality gates (tests/CI/security), and collaboration signals. I'll share a weekly evidence summary so nothing relies on memory."

(Anchors to multidimensional productivity guidance. ³⁹)

When you suspect low effort

"I'm seeing repeated patterns: large changes without sufficient tests, slow response to review feedback, and no clear de-risking narrative. Let's agree on two deliverables and the quality gates for the next two weeks, and I'll remove blockers where I can."

Starting the improvement plan

“This plan isn’t about code output. It’s about meeting the role’s expectations: predictable delivery, quality, and ownership. Each week we’ll review an evidence pack from tickets/PRs/CI, agree next steps, and track pass/fail criteria.”

Low-effort documentation practices

- Put expectations and weekly goals **in writing** (ticket comment or shared doc). CIPD emphasizes performance management is supported by recording objectives and improvement plans, but the main focus is regular performance discussions—so keep the writing lightweight and continuous. ⁴⁰
- Use AI to draft weekly summaries, but require manager edits (“human-in-loop”) to avoid automation bias and accuracy errors. ⁴¹
- Keep a single “decision log” for the plan: what was expected, what happened, what support was provided, what changed.

Metrics and charts to monitor progress (what to graph)

Use charts to answer: **Is the person converging? Are they reducing risk? Are they becoming predictable?**

Recommended weekly charts (per person, then compared to team medians within the same difficulty band):

- **Delivery predictability:** planned vs done (count of commitments met) separated by difficulty band.
- **PR reviewability:** median PR size (diff size) and review cycle time (open → first review → merge).
- **Rework:** % of PRs requiring follow-up “fix” PR within 7 days; revert/hotfix count tied to changes.
- **Quality gates:** CI pass rate on merge; test coverage deltas on touched modules; code scanning alert trend. ⁴²
- **Collaboration:** review turnaround time and number of substantive review comments (qualitative spot checks).
- **Operational impact:** incidents or pages tied to their changes and time-to-mitigate involvement.

If you already track DORA metrics, use them primarily at **team/service level** to ensure the environment supports success; DORA’s own materials explain metric definitions and evolution (and warn against simplistic use). ⁴³

¹ ⁴ <https://www.microsoft.com/en-us/research/publication/the-impact-of-ai-on-developer-productivity-evidence-from-github-copilot/>

<https://www.microsoft.com/en-us/research/publication/the-impact-of-ai-on-developer-productivity-evidence-from-github-copilot/>

² ⁶ ¹³ ³⁹ <https://queue.acm.org/detail.cfm?id=3454124>

<https://queue.acm.org/detail.cfm?id=3454124>

³ ²² ⁴⁰ <https://www.cipd.org/en/knowledge/factsheets/performance-factsheet/>

<https://www.cipd.org/en/knowledge/factsheets/performance-factsheet/>

⁵ https://economics.mit.edu/sites/default/files/inline-files/draft_copilot_experiments.pdf

https://economics.mit.edu/sites/default/files/inline-files/draft_copilot_experiments.pdf

- 7 38 https://link.springer.com/chapter/10.1007/978-1-4842-4221-6_2
https://link.springer.com/chapter/10.1007/978-1-4842-4221-6_2
- 8 <https://www.damtp.cam.ac.uk/user/mem/papers/LHCE/goodhart.html>
<https://www.damtp.cam.ac.uk/user/mem/papers/LHCE/goodhart.html>
- 9 <https://www.microsoft.com/en-us/research/publication/taking-flight-with-copilot-early-insights-and-opportunities-of-ai-powered-pair-programming-tools/>
<https://www.microsoft.com/en-us/research/publication/taking-flight-with-copilot-early-insights-and-opportunities-of-ai-powered-pair-programming-tools/>
- 10 19 <https://cacm.acm.org/research-highlights/asleep-at-the-keyboard-assessing-the-security-of-github-copilots-code-contributions/>
<https://cacm.acm.org/research-highlights/asleep-at-the-keyboard-assessing-the-security-of-github-copilots-code-contributions/>
- 11 16 36 41 <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>
<https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>
- 12 <https://www.cs.umd.edu/~mvz/handouts/gqm.pdf>
<https://www.cs.umd.edu/~mvz/handouts/gqm.pdf>
- 14 20 43 <https://dora.dev/guides/dora-metrics/>
<https://dora.dev/guides/dora-metrics/>
- 15 <https://dora.dev/research/2025/dora-report/>
<https://dora.dev/research/2025/dora-report/>
- 17 <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/reviewing-changes-in-pull-requests/about-pull-request-reviews>
<https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/reviewing-changes-in-pull-requests/about-pull-request-reviews>
- 18 <https://docs.github.com/en/copilot/how-tos/use-copilot-for-common-tasks/create-a-pr-summary>
<https://docs.github.com/en/copilot/how-tos/use-copilot-for-common-tasks/create-a-pr-summary>
- 21 37 <https://nvlpubs.nist.gov/nistpubs/ir/2021/NIST.IR.8312.pdf>
<https://nvlpubs.nist.gov/nistpubs/ir/2021/NIST.IR.8312.pdf>
- 23 <https://www.ilo.org/algorithmic-management-workplace>
<https://www.ilo.org/algorithmic-management-workplace>
- 24 <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.1270.pdf>
<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.1270.pdf>
- 25 33 <https://docs.github.com/en/copilot/concepts/copilot-usage-metrics/copilot-metrics>
<https://docs.github.com/en/copilot/concepts/copilot-usage-metrics/copilot-metrics>
- 26 32 <https://docs.github.com/en/code-security/concepts/code-scanning/copilot-autofix-for-code-scanning>
<https://docs.github.com/en/code-security/concepts/code-scanning/copilot-autofix-for-code-scanning>
- 27 <https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-code-owners>
<https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-code-owners>

28 <https://docs.github.com/articles/enabling-required-status-checks>

<https://docs.github.com/articles/enabling-required-status-checks>

29 42 <https://docs.github.com/code-security/code-scanning/automatically-scanning-your-code-for-vulnerabilities-and-errors/about-code-scanning-with-codeql>

<https://docs.github.com/code-security/code-scanning/automatically-scanning-your-code-for-vulnerabilities-and-errors/about-code-scanning-with-codeql>

30 <https://docs.github.com/en/code-security/concepts/secret-security/about-push-protection>

<https://docs.github.com/en/code-security/concepts/secret-security/about-push-protection>

31 <https://docs.github.com/en/copilot/concepts/agents/code-review>

<https://docs.github.com/en/copilot/concepts/agents/code-review>

34 <https://dora.dev/resources/>

<https://dora.dev/resources/>

35 <https://devlake.apache.org/docs/DORA/>

<https://devlake.apache.org/docs/DORA/>